

Interaction and Forms

Building clear navigation and accessible forms — and proving, with evidence, that focus order, labels, errors and keyboard access all work.



Technical outcomes

By the end of this lecture you can build, test and defend an accessible interactive control.

Build

Choose native controls first; label every one; group related choices with fieldset and legend.

Test

Drive every control with only a keyboard; confirm focus order, visible focus and screen-reader announcement.

Prove

Record what you tested as evidence — the interaction map this week produces feeds Assignment 3 directly.

Why this week is graded work, not theory

Assignment 3 — Accessible interaction prototype — 25%, due 5:00 pm Friday 7 May 2027.

Assignment 3 criterion	Built directly from
Task flow and interaction clarity	This week's navigation and form-structure work
Accessible forms and keyboard use	Labels, fieldset/legend, focus order, visible focus
Visual feedback and technical care	Error states, live regions, colour-independent feedback
Testing evidence and revision	The keyboard test protocol and interaction map, slides 26–28

Two ways to build any control

A native element or a custom widget built from generic markup — the choice has consequences.

Native element

`<button>`, `<input>`, `<select>`, `<a href>`. The browser supplies role, name, state, keyboard behaviour and focus handling for free.

Custom widget

A `<div>` or `<span>` styled to look like a control. None of role, keyboard behaviour or state exist until you add every one yourself with ARIA and script.

First Rule of ARIA (W3C WAI-ARIA Authoring Practices): if a native HTML element has the semantics and behaviour you need, use it instead of repurposing an element and adding ARIA.

When ARIA is actually warranted

A short decision method, not a reflex reach for aria-* attributes.

- 1 Can this be a native element (button, input, select, a, details)? If yes, use it and stop.
- 2 Is there no native equivalent (a tablist, a combobox, a tree)? Only then look at the ARIA Authoring Practices Guide pattern for it.
- 3 Copy that pattern's exact role, state attributes and keyboard behaviour — not just the role.
- 4 Test with a keyboard and a screen reader before shipping. ARIA that is not tested is often worse than no ARIA.

Bad ARIA is worse than no ARIA: it tells assistive technology a promise (“this is a working tab list”) that the script does not keep.

One semantic contract, many input devices

Correct roles and keyboard behaviour are what let radically different devices operate the same control.

A mouse click, a keyboard Enter, a switch-scan selection and a screen-reader “activate” gesture are all different inputs. A correctly built control responds to all of them because it exposes one role and one activation behaviour — not four separate implementations.

Practical test



Labels — three ways to name a control

aria-label and aria-labelledby exist for edge cases, not as a substitute for a visible label.

```
<!-- Preferred: explicit label, visible and clickable -->
<label for="email">Email address</label>
<input id="email" type="email" autocomplete="email">

<!-- No visible text available (icon-only button) -->
<button aria-label="Close dialog">X</button>

<!-- Label built from existing visible text elsewhere -->
<h2 id="billing">Billing address</h2>
<input aria-labelledby="billing" name="address1">
```

Rule of thumb: use a visible `<label for>` by default. Reach for `aria-label` only when no visible text can serve as the name; use `aria-labelledby` to reuse text that already exists on the page.

Placeholders are not labels

A placeholder is a hint that disappears; a label is a permanent, programmatically associated name.

```
<!-- Fails: placeholder only -->
<input type="text" placeholder="Full name">

<!-- Passes: label kept, placeholder optional -->
<label for="fullname">Full name</label>
<input id="fullname" type="text"
  placeholder="e.g. Priya Shah">
```

Vanishes on input

Placeholder text disappears when the input is focused.

Inconsistent contrast and exposure

Placeholder text is not visible when the input is focused.

Grouping related choices — fieldset and legend

A screen reader announces the group's name before it announces each option inside it.

```
<fieldset>
  <legend>Preferred contact method</legend>
  <label><input type="radio" name="contact" value="email"> Email</label>
  <label><input type="radio" name="contact" value="phone"> Phone</label>
  <label><input type="radio" name="contact" value="post"> Post</label>
</fieldset>
```

Without fieldset/legend, a screen reader reads three unrelated radio buttons named “Email”, “Phone”, “Post” with no shared question. With it, each announces as “Preferred contact method, Email, radio button, 1 of 3”.

Use fieldset/legend for any radio group, checkbox group, or a short run of related inputs (e.g. a date split into day/month/year).

Semantics survive the input device

Switch-scanning access relies on the same roles and tab order as keyboard and screen-reader use.

A single-switch user does not press keys directly. Software scans through focusable elements in order and the user presses one switch at the right moment to select the highlighted item.

What breaks this

What this week fixes



Focus order — DOM order, not visual order

Tab order follows the HTML source order. CSS can move things visually without moving them logically.

```
<!-- tabindex pitfalls -->
tabindex="0"    /* joins natural tab order — fine */
tabindex="-1"   /* focusable only by script — fine, e.g. skip
target */
tabindex="3"    /* positive value: rewrites tab order — avoid */
```

If a CSS grid or flexbox reorder makes the visual layout diverge from the HTML order, keyboard users tab through the page in an order that no longer matches what they see.

Fix the HTML order first. Reach for a positive tabindex only as a last resort — it is fragile and rarely necessary.



Visible focus — and the outline: none anti-pattern

Removing the focus ring without replacing it hides where keyboard attention is on the page.

```
/* Anti-pattern: focus becomes invisible for every user */
button:focus { outline: none; }

/* Correct: hide the ring only for pointer users, keep it for keyboard */
button:focus:not(:focus-visible) { outline: none; }
button:focus-visible {
  outline: 3px solid #C7831A;
  outline-offset: 2px;
}
```

:focus-visible lets the browser show a focus ring for keyboard and switch users while suppressing it after a mouse click — without hiding it altogether.

Never ship outline: none alone. Test every interactive element by tabbing to it and checking the ring is visible against its background.

Keyboard-only is the default test, not an edge case

Unplug the mouse before you call an interaction finished.

Who relies on this

People with motor impairments using only a keyboard or switch, screen-reader users, and power users — keyboard access is not a minority accommodation, it is baseline correctness.

The habit to build now

Before checking a form into the studio task, tab through it start to finish with your hand off the mouse.



Skip links — the first tab stop on any page

Let a keyboard user bypass repeated navigation instead of tabbing through it on every page.

```
<body>
  <a class="skip-link" href="#main">Skip to main content</a>
  <nav>... 12 links ...</nav>
  <main id="main">...</main>
</body>

.skip-link {
  position: absolute; left: -999px;
}
.skip-link:focus { left: 1rem; top: 1rem; } /* becomes visible on focus */
```

The link must be the first focusable element and must become visible on focus — hiding it with `display:none` removes it from the tab order entirely.

Disclosure pattern — aria-expanded

A toggle button must announce its own current state.

```
<button aria-expanded="false" aria-controls="filters-panel" id="filters-btn">
  Show filters
</button>
<div id="filters-panel" hidden>... filter controls ...</div>

// On click: toggle the hidden attribute and flip aria-expanded to match
btn.addEventListener("click", () => {
  const open = panel.hasAttribute("hidden");
  panel.toggleAttribute("hidden", !open);
  btn.setAttribute("aria-expanded", String(open));
});
```

Modal dialog — trapping and returning focus

Four behaviours a modal must get right, or it breaks keyboard use entirely.

On open

Move focus to the dialog (its heading or first control).

While open

Tab and Shift+Tab cycle only through elements inside the dialog.

Escape

Closes the dialog without requiring a pointer.

On close

Focus returns to the element that opened it, not to the top of the page.

Semantics are read, not just seen

Every role, label and state you set is consumed by assistive technology as data, not as a visual hint.

A refreshable Braille display renders the accessibility tree's exposed name, role and state as raised pin patterns — there is no visual layout to fall back on if the underlying markup is wrong.

What this means for you

A control that “looks right” in a screenshot can still be unusable here if its role or name is missing — test the tree, not just the screen.



Accessible error handling — connect the message to the field

aria-describedby links help and error text directly to the control that owns it.

```
<label for="email">Email address</label>
<input id="email" type="email"
      aria-describedby="email-error"
      aria-invalid="true">
<p id="email-error" role="alert">
  Enter an email address in the form name@example.com
</p>
```

aria-describedby makes the screen reader announce the error immediately after the field name. aria-invalid marks the field's state; role="alert" announces new error text as soon as it appears.

Error summaries — and wording that tells someone what to do

A red border communicates nothing to someone who cannot see colour, and nothing at all to a screen reader.

```
<div role="alert" class="error-summary">
  <h2>2 problems need fixing</h2>
  <ul>
    <li><a href="#email">Email address is missing</a></li>
    <li><a href="#password">Password must be at least 8 characters</a></li>
  </ul>
</div>
```

Vague (avoid)

“Invalid input.” — does not say which field, or what a valid value looks like.

Actionable (use)

“Enter a password of at least 8 characters.” — names the field, the rule, and how to fix it.

Live regions — announcing change without moving focus

aria-live tells a screen reader to announce new or changed content automatically.

Attribute	Behaviour and use
aria-live="polite"	Waits for the user to pause. Use for status text, save confirmations, result counts.
aria-live="assertive"	Interrupts immediately. Reserve for time-critical errors, e.g. a form submission failure.
role="status"	Shorthand for a polite live region; commonly used for “Saving...” / “Saved” messages.
role="alert"	Shorthand for an assertive live region; the pattern used on the previous two slides.

Announcing across every channel at once

A change that is only visible fails everyone who is not looking at that exact spot when it happens.

An interpreter renders one message through a second channel simultaneously, without waiting to be asked. A live region does the same job for a web page — the same update reaches a screen reader the instant it reaches the screen.

Design rule



Before and after — the same form, two implementations

Identical appearance is possible with wildly different accessibility.

```
<!-- div-soup: name only, no role, no keyboard support -->
<div class="field">
  <div class="field-label">Email</div>
  <div class="input" contenteditable="true"></div>
</div>
<div class="btn" onclick="submit()">Submit</div>
```

```
<!-- native: name, role, keyboard behaviour for free -->
<label for="email">Email</label>
<input id="email" type="email" required>

<button type="submit">Submit</button>
```

The left version has no accessible name, no keyboard activation and no submit semantics. The right version gets all three from the browser, at zero extra script.

Visual feedback needs more than colour

WCAG 1.4.1 requires that a state or error is identifiable without relying on colour alone.

A red border alone fails for a colour-blind user, and can fail entirely at high magnification if it falls outside the zoomed viewport.



Pair colour with

Also check

Text reflows and stays readable at 200% browser zoom, and every state has at least 3:1 contrast against its background (WCAG 1.4.11).

Testing in the browser — the accessibility tree

DevTools exposes the same name, role and state a screen reader receives.

- 1 Open the browser's Accessibility panel (DevTools) alongside the Elements panel.
- 2 Select a control and confirm its computed name, role and state match what you intended.
- 3 Confirm the tab order in the panel matches the visual reading order.



The keyboard test protocol

A fixed sequence to run on every interactive page before it is considered done.

- 1 Disconnect or ignore the mouse. Press Tab from the top of the page.
- 2 Confirm a visible focus indicator appears on every stop, with no silent jumps.
- 3 Confirm the order matches the visual reading order, left-to-right, top-to-bottom.
- 4 Activate each control with Enter (links, buttons) or Space (buttons, checkboxes).
- 5 Press Shift+Tab to confirm the same order reverses cleanly.
- 6 Press Escape on any open menu, disclosure or dialog and confirm it closes.

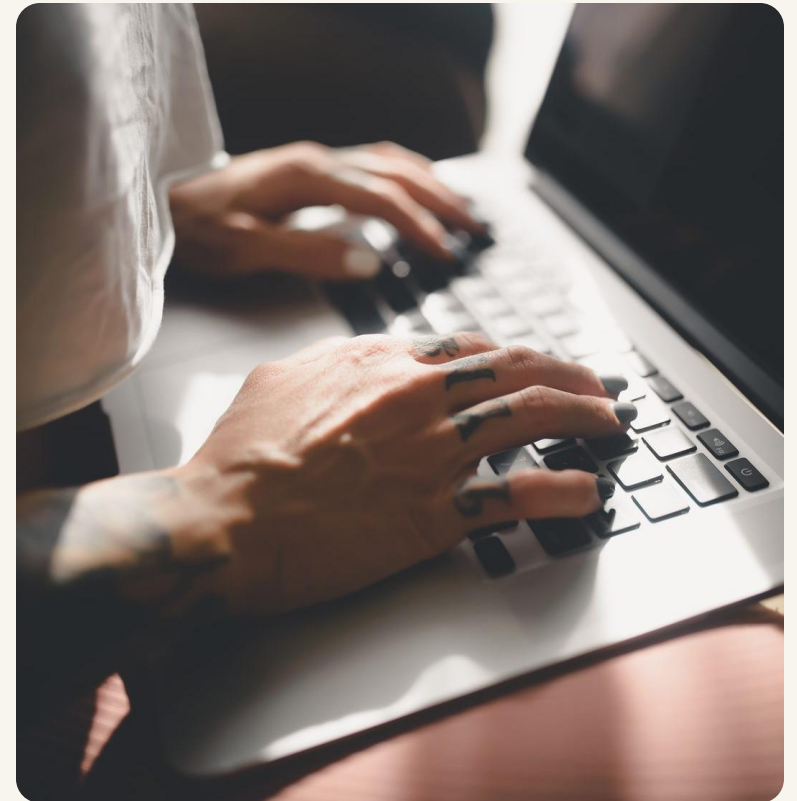
Recording what you actually tested

Evidence, not memory, is what Assignment 3's testing criterion looks for.

A verbal claim (“I checked keyboard access”) is not evidence. A dated record of what you pressed, what you expected and what actually happened is.

Keep for every control

Its name as announced, its role, its position in tab order, and the result of pressing Enter/Space/Escape where relevant.



The interaction map

One row per control — the artefact this week's testing protocol produces.

Control	Expected role	Tab position	Key test	Result
Submit button	button	7 of 9	Space activates; focus ring visible	Pass — recorded 2027-04-...

Build this table for every interactive control in your studio task; it is the direct evidence Assignment 3's rubric asks for.

Studio task — build the Assignment 3 bridge piece

What you build and test in this session.

Brief

Build a short navigation (with a skip link) and one accessible form: at least one text input, one grouped radio or checkbox set, and a submit action with error handling.

Constraints

Native elements only unless you can justify a custom widget against the First Rule of ARIA on slide 4. Every control must be operable by keyboard alone.

Evidence to submit

An interaction map (slide 28's template) covering every control, plus one paragraph on the keyboard test you ran and what you fixed as a result.

Review rubric

How this week's interaction and form work is assessed against Assignment 3's criteria.

Task flow and interaction clarity

Navigation and form order are predictable and match visual layout.

Accessible forms and keyboard use

Every control has a name, a role, and works with keyboard alone.

Visual feedback and technical care

Errors and state changes are visible, described in text, and announced to assistive technology.

Testing evidence and revision

An interaction map is present and shows at least one fix made after testing.

Source records and further practice

Every photograph in this deck, and where to read the underlying standards.

Image sources (Wikimedia Commons, all CC0/CC-BY/CC-BY-SA, accessed 13 Sep 2026)

- Wikimedia Commons: File:Backlit keyboard.jpg — https://commons.wikimedia.org/wiki/File:Backlit_keyboard.jpg — CC BY-SA 4.0 — accessed 13 Sep 2026

- Wikimedia Commons: File:Single switch onscreen keyboard.jpg (photographer vtsaran) — https://commons.wikimedia.org/wiki/File:Single_switch_onscreen_keyboard.jpg — CC BY 2.0 — accessed 13 Sep 2026

- Wikimedia Commons: File:Artwork of typing using a computer keyboard.jpg — https://commons.wikimedia.org/wiki/File:Artwork_of_typing_using_a_computer_keyboard.jpg — CC BY-SA 4.0 — accessed 13 Sep 2026

- Wikimedia Commons: File:Sign language interpreter, 2012 (01).jpg — [https://commons.wikimedia.org/wiki/File:Sign_language_interpreter,_2012_\(01\).jpg](https://commons.wikimedia.org/wiki/File:Sign_language_interpreter,_2012_(01).jpg) — CC BY-SA 2.0 — accessed 13 Sep 2026

- Wikimedia Commons: File:Code on computer monitor (Unsplash).jpg (photographer Markus Spiske) — [https://commons.wikimedia.org/wiki/File:Code_on_computer_monitor_\(Unsplash\).jpg](https://commons.wikimedia.org/wiki/File:Code_on_computer_monitor_(Unsplash).jpg) — CC0 — accessed 13 Sep 2026

- Wikimedia Commons: File:Keyboard with fingers.jpg — https://commons.wikimedia.org/wiki/File:Keyboard_with_fingers.jpg — CC BY-SA 4.0 — accessed 13 Sep 2026

- Wikimedia Commons: File:Hands are typing on a laptop keyboard while a clock rests next to the device.jpg — https://commons.wikimedia.org/wiki/File:Hands_are_typing_on_a_laptop_keyboard_while_a_clock_rests_next_to_the_device.jpg — CC BY 2.0 — accessed 13 Sep 2026

- Wikimedia Commons: File:Refreshable Braille display.jpg — https://commons.wikimedia.org/wiki/File:Refreshable_Braille_display.jpg — CC BY-SA 3.0 — accessed 13 Sep 2026

- Wikimedia Commons: File:WTBBL magnifier 01.jpg — https://commons.wikimedia.org/wiki/File:WTBBL_magnifier_01.jpg — CC BY-SA 3.0 — accessed 13 Sep 2026

- Wikimedia Commons: File:Hands are seen typing on a laptop keyboard.jpg — https://commons.wikimedia.org/wiki/File:Hands_are_seen_typing_on_a_laptop_keyboard.jpg — CC BY 2.0 — accessed 13 Sep 2026

Standards and further reading

W3C WAI-ARIA Authoring Practices Guide (w3.org/WAI/ARIA/apg) · WebAIM Forms tutorial (webaim.org/techniques/forms) · MDN Accessibility docs (developer.mozilla.org/en-US/docs/Web/Accessibility). Accessed 13 September 2026.